

Entrenar una red neuronal

01

Epoch

02

Batch

03

Callback

04

Overfitting

Ya conocéis la operación que vive dentro de una neurona

NumPy

$$z = X @ w + b$$

X contiene ejemplos y variables.

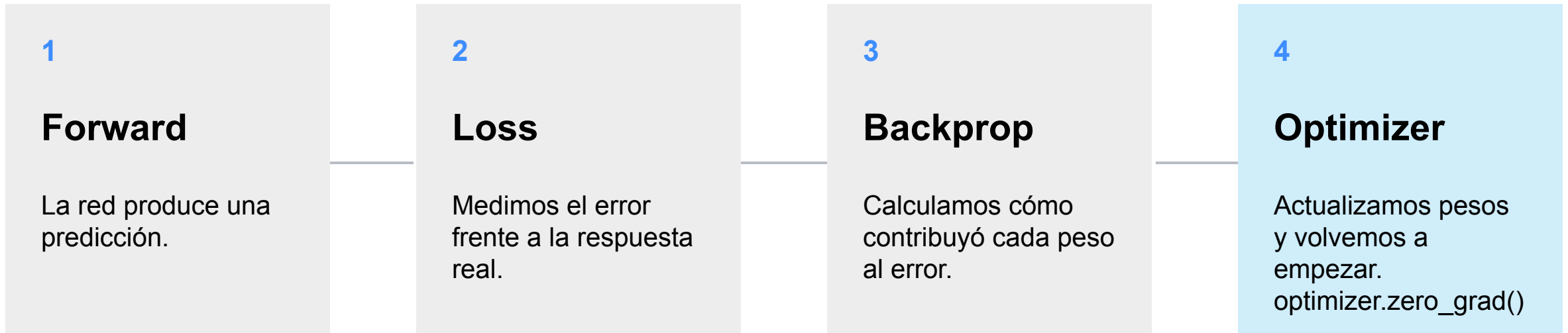
w contiene los pesos.

b desplaza la decisión.

Una red neuronal añade capas y funciones de activación, pero la idea central sigue siendo transformar entradas con pesos aprendidos.

Conexión clave: producto punto → neurona → capa → red.

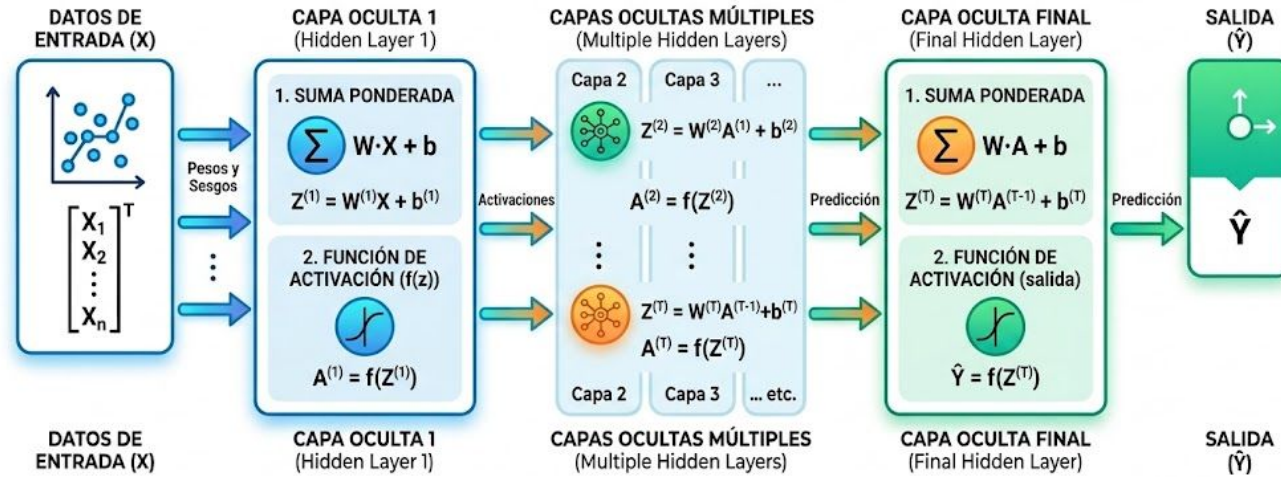
Entrenar es repetir un bucle de feedback



El aprendizaje ocurre cuando el error vuelve hacia atrás y modifica los pesos.

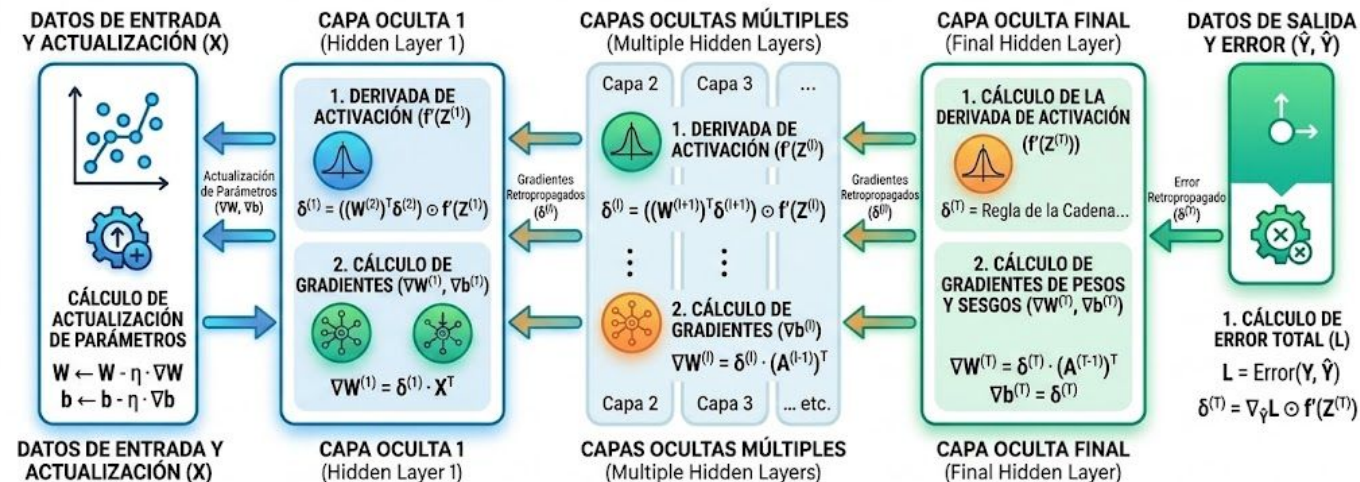
FOROWARD PROPAGATION EN UNA RED NEURONAL (NEURIRAL NETWORK)

FLUJO DE INFORMACIÓN (DATOS)



BACKPROPAGATION EN UNA RED NEURONAL (NEURIRAL NETWORK)

FLUJO DE GRADIENTES Y ERRORES



REGLA DE LA CADENA (CHAIN RULE)

Batch, iteración y epoch miden escalas diferentes

Batch

Un grupo pequeño de ejemplos que entra a la red a la vez.

Ej.: 32 muestras

Iteración

Una actualización de pesos después de procesar un batch.

1 batch → 1 actualización

Epoch

Una pasada completa por todos los datos de entrenamiento.

$N \text{ muestras} \div \text{batch_size}$

Con 1.000 muestras y batch_size=100: hay 10 iteraciones por epoch.

Qué se hace a nivel de BATCH

(En cada lote de datos)

Estas operaciones ocurren miles de veces por época. Son puramente matemáticas y de optimización.

- **Paso hacia adelante (*Forward pass*):** El modelo procesa un pequeño subconjunto de datos y genera predicciones.
- **Cálculo de la pérdida local (*Batch Loss*):** Se mide el error cometido únicamente en los datos de ese lote.
- **Paso hacia atrás (*Backward pass / Backpropagation*):** Se calculan los gradientes (dirección del error) para saber cómo ajustar los pesos.
- **Actualización de pesos (*Weight update*):** El optimizador (como Adam o SGD) modifica los parámetros del modelo inmediatamente.
- **Actualización de métricas de entrenamiento:** Se acumula el progreso del *loss* y el *accuracy* para mostrar la barra de carga en consola.

Qué se hace a nivel de EPOCH

(Al terminar de ver todos los datos)

Estas operaciones ocurren solo una vez al final de una vuelta completa a todo el conjunto de datos.

- **Evaluación en validación:** El modelo se congela (no aprende) y procesa el conjunto de `val_loss` y `val_accuracy`.
- **Ejecución del Early Stopping:** El sistema revisa si el error de validación mejoró o si debe detener el entrenamiento.
- **Guardado de checkpoints:** Se salva en el disco duro el estado del modelo si ha alcanzado su mejor rendimiento histórico.
- **Ajuste del Learning Rate (Scheduler):** Se reduce la velocidad de aprendizaje si el modelo se ha estancado.
- **Mezcla de datos (*Shuffling*):** Se desordena todo el conjunto de entrenamiento para que los lotes del siguiente epoch sean diferentes y el modelo no memorice el orden.

Compile define el problema; fit ejecuta el aprendizaje

model.compile(...)

```
optimizer='adam'  
loss='binary_crossentropy'  
metrics=['accuracy']
```

Define cómo aprender y cómo medir.

model.fit(...)

```
X_train, y_train  
epochs=100  
batch_size=32  
validation_split=0.2  
callbacks=[...]
```

Ejecuta el bucle y devuelve history.

Training y validation responden preguntas distintas

TRAINING

¿Está aprendiendo los ejemplos que le mostramos?

Aquí se actualizan los pesos.

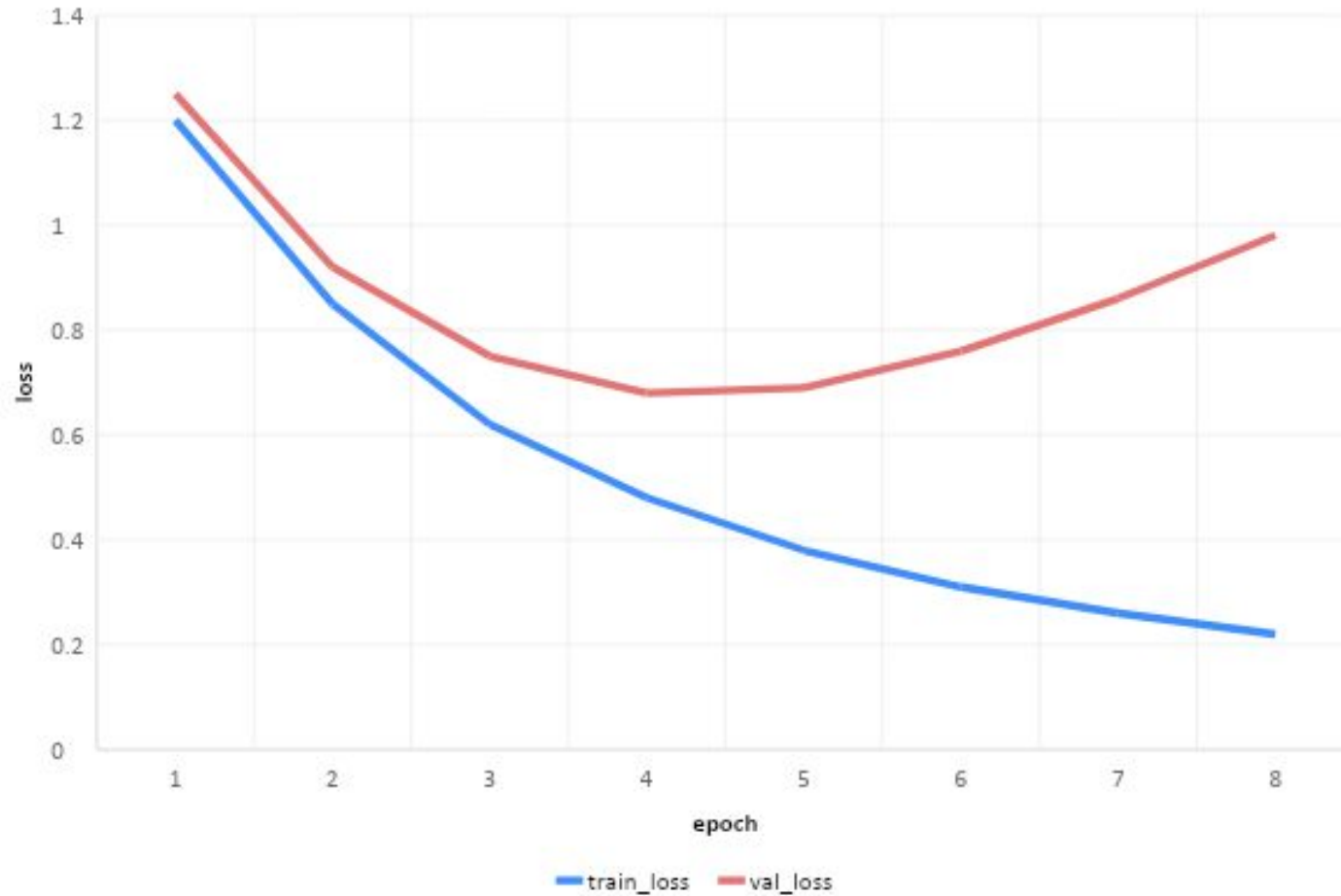
VALIDATION

¿Funciona también con ejemplos que no usa para aprender?

Aquí no se actualizan los pesos.

Una mejora en training no garantiza una mejora fuera de training.

El overfitting aparece cuando las curvas se separan



Señal

**train_loss baja
val_loss deja de bajar y
sube**

**La mejor epoch no suele
ser la última.**

Early Stopping y Dropout resuelven problemas distintos

Early Stopping

Observa val_loss.
Detiene el entrenamiento cuando deja de mejorar.
Puede restaurar los mejores pesos.

Controla cuándo parar.

Dropout

Durante training apaga aleatoriamente una fracción de activaciones de las neuronas.
Obliga a la red a no depender de un único camino.

Cambia cómo aprende.

Los callbacks convierten el entrenamiento en un proceso controlado

EarlyStopping

Parar cuando val_loss no mejora. (Patience)

ModelCheckpoint

Guardar la mejor versión del modelo.

ReduceLROnPlateau

Bajar el learning rate cuando el avance se estanca.

Un entrenamiento serio no solo aprende: también observa, guarda y decide.

Un bloque de Keras contiene toda la historia

```
model = keras.Sequential([
    layers.Dense(16, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=5,
    restore_best_weights=True)
```

```
history = model.fit(
    X_train, y_train,
    epochs=100, batch_size=32,
    validation_split=0.2,
    callbacks=[stop])
```

Dense

Transforma

Dropout

Regulariza

compile

Define

fit

Entrena

history

Cuenta qué pasó

Si podéis explicar el bucle, se puede depurar el modelo

Predicción → loss → gradientes →
actualización → validación → decisión

NOTEBOOK EXPLICADO + ACTIVIDAD DIDÁCTICA

Leer con detalle los notebook en este orden:

- 1: Punto de partida
- 2: Maneras de programar una red neuronal

Ve a esta página web

<https://playground.tensorflow.org/>

Seleccionar una entrada de datos y juega a cuántos neuronas necesitas para que se separen los datos.

Después intenta comprender el qué learning rate usasteis, qué función de activación, cuántas neuronas en cada capa, etc

Notebook 1: "Punto_de_partida.ipynb" (La IA desde cero)

Este notebook es el laboratorio matemático. Aquí verás qué hay "bajo el capó" de una red neuronal sin que ninguna herramienta externa les haga el trabajo.

- **1. El problema a resolver (Dataset Artificial):** Creación y visualización de un conjunto de datos complejo en forma de círculos concéntricos (`make_circles`), ideal para demostrar que una línea recta no basta para clasificar datos en el mundo real.
- **2. Arquitectura interna de las neuronas:** Programación manual de la estructura de la red, definiendo la clase `neural_layer` y construyendo las funciones matemáticas de activación como la Sigmoid y la ReLU.
- **3. El motor de aprendizaje al descubierto:** Implementación desde cero de la función de entrenamiento, donde se desglosan paso a paso el *Forward Pass*, el *Backpropagation* (cálculo del error) y el *Gradient Descent* (actualización de pesos y sesgos).
- **4. Renderizado visual en tiempo real:** Uso de un bucle de entrenamiento que, gracias a bibliotecas gráficas, muestra una animación *frame a frame* de cómo la inteligencia artificial va ajustando su mapa de predicción hasta rodear los datos perfectamente.

Notebook 2: "3 Maneras de Programar a una Red Neuronal" (Herramientas de la Industria)

Una vez que has entendido la teoría en el primer notebook, este segundo documento te demuestra cómo el mercado laboral resuelve el mismo problema de los círculos concéntricos de tres formas diferentes, optimizando el tiempo y el código.

- **1. TensorFlow (Bajo nivel dinámico):** Definición explícita de variables y uso de la herramienta `GradientTape` para que la librería calcule las derivadas automáticamente por ellos en cada iteración.
- **2. Keras (La API de alto nivel):** Transición a la sintaxis moderna, donde crear la arquitectura se resume a apilar bloques con `Sequential` y añadir capas `Dense` en unas pocas líneas.
- **3. Control de ciclo de vida y sobreajuste (Overfitting):** Implementación de métricas de validación, "apagones" aleatorios de neuronas (`Dropout`) y asistentes automáticos (*Callbacks* como el *Early Stopping* y el ajuste dinámico del *Learning Rate*) que programamos en pasos anteriores.
- **4. Scikit-Learn (El enfoque tradicional):** Uso del clasificador `MLPRegressor` para resolver el mismo problema tratándolo como una "caja negra", configurando los parámetros directamente en una sola instrucción sin necesidad de construir la red capa a capa.

¡No te olvides de la actividad didáctica!

<https://playground.tensorflow.org/>